PROJECT DOCUMENTATION

InvestoraAI

A full-stack AI-powered financial intelligence dashboard with multi-user authentication, real-time market sentiment analysis, automated alerts, and weekly report generation.

React + Vite + TypeScript | FastAPI + LangChain | n8n Workflow Automation | OpenAI GPT-4 | Pinecone Vector DB | Telegram Alerts | Notion Integration | Docker

InvestoraAI · Version 1.0 · March 2026 · Full Stack Financial AI Agent

01 Project Overview // what is it?

InvestoraAI is an AI-powered financial intelligence platform that continuously monitors financial markets, analyzes news sentiment using Large Language Models, stores insights in a vector database, and delivers personalized alerts to investors via Telegram and Notion.

The platform combines a modern React dashboard with a Python AI backend orchestrated through n8n — a low-code automation tool running in Docker. Every user gets a personal profile with configurable risk tolerance, market interests, and alert preferences.

Real-time	Multi-user	GPT-4	Weekly
MARKET ANALYSIS	AUTH SYSTEM	SENTIMENT ENGINE	AUTO REPORTS

02 Tech Stack & Tools // technologies used

FRONTEND

React 18	UI framework with hooks
Vite	Lightning-fast dev server + bundler
TypeScript	Type-safe JavaScript
Tailwind CSS	Utility-first CSS framework
shadcn/ui	Accessible component library
Framer Motion	Animations & transitions
React Router v6	Client-side routing
TanStack Query	Async state management

BACKEND (FASTAPI)

FastAPI	High-performance Python API
LangChain	LLM orchestration framework
LangGraph	ReAct agent state machine
OpenAI GPT-4	Sentiment analysis + reports
text-embedding-3-small	Article embeddings
Pinecone	Vector database for RAG
Uvicorn	ASGI server
Python-dotenv	Environment config

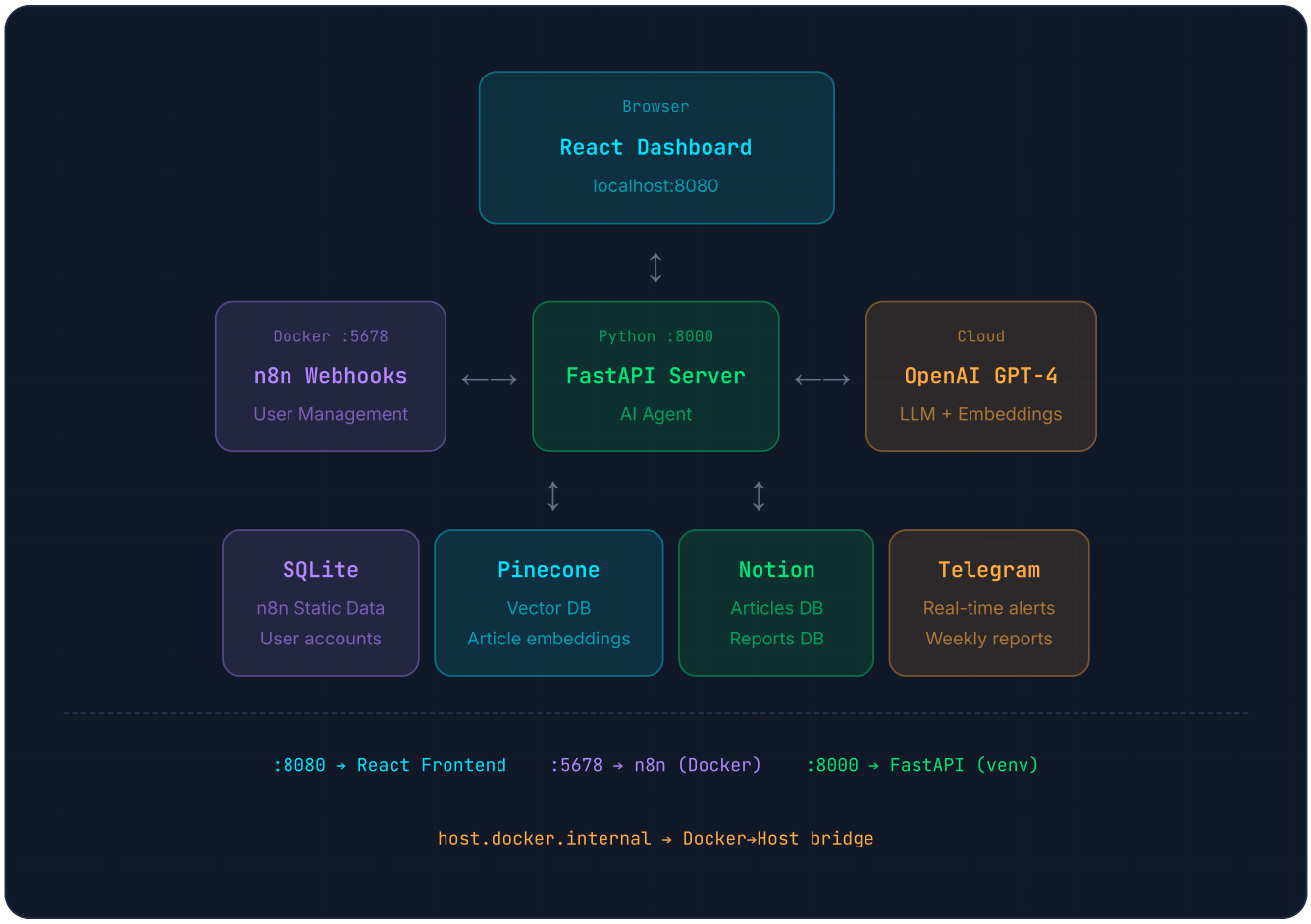
AUTOMATION (N8N)

n8n v2.9+	Self-hosted workflow automation
Docker	Container for n8n runtime
Webhook nodes	HTTP trigger endpoints
Code nodes	JavaScript logic in n8n
Static Data	SQLite-backed key-value store
Schedule Trigger	Cron-based execution

EXTERNAL SERVICES

News API	Financial news headlines
Alpha Vantage	Market data + sentiment
Finnhub	Real-time market news
Telegram Bot	Push alert delivery
Notion API	Article + report storage
OpenAI API	GPT-4 + embeddings
Pinecone Cloud	Vector similarity search

03 System Architecture // how everything connects



KEY NETWORKING NOTE n8n runs inside Docker so it cannot use `localhost` to reach FastAPI. Instead it uses `host.docker.internal:8000` which is Docker's built-in bridge to the host machine's network.

04 Multi-User Authentication // register · login · session

User accounts are stored in **n8n's workflow static data** (persisted to SQLite inside Docker). Passwords are hashed client-side using SHA-256 before ever leaving the browser. No plain-text passwords are transmitted or stored anywhere.

📝 REGISTRATION FLOW

1. **User fills Register form**
   Username, password, confirm password, risk tolerance, market interests, Telegram Chat ID
2. **Client-side password hashing**
   Browser runs `SHA-256(password + "investora_2026")` — password never sent in plain text
3. **POST → n8n Register Webhook**
   `localhost:5678/webhook/investora/register` with username, hash, UUID, profile
4. **n8n Code node stores user**
   Saves to `$getWorkflowStaticData('global').users` in SQLite — persists forever
5. **Auto-login fires**
   Immediately calls the login webhook with same credentials
6. **Session → localStorage**
   Stores `{ username, userId, profile }` locally. Navigates to dashboard.

🔑 LOGIN & SESSION FLOW

1. **User fills Login form**
   Username + password
2. **Same hash computed**
   `SHA-256(password + "investora_2026")`
3. **POST → n8n Login Webhook**
   `localhost:5678/webhook/investora/login`
4. **n8n verifies hash**
   Compares stored hash === incoming hash. Returns userId + profile if match.
5. **Session rehydration**
   On browser refresh, session is read from `localStorage` — no network call needed
6. **ProtectedRoute guard**
   If no session → redirected to `/login`. Logged in users see dashboard only.

N8N USER MANAGEMENT WEBHOOK ENDPOINTS

ENDPOINT	METHOD	PAYLOAD	RESPONSE
/webhook/investora/register	POST	username, passwordHash, userId, profile	{ ok, userId } or { ok:false, error }
/webhook/investora/login	POST	username, passwordHash	{ ok, userId, profile } or { ok:false }
/webhook/investora/profile	POST	username, profile object	{ ok: true }

05 n8n Financial Agent Workflow // automation pipeline

The main financial workflow runs on a schedule, fetching news from three sources, passing them to the FastAPI AI agent for sentiment analysis, then routing results to Telegram and Notion simultaneously.

WORKFLOW NODE STATUS (DASHBOARD DISPLAY)

• RUNNING News API Fetch 2 min ago	✓ SUCCESS Alpha Vantage 2 min ago	✓ SUCCESS Finnhub Market 2 min ago	• RUNNING LangChain ReAct 1 min ago
✓ SUCCESS Pinecone RAG 1 min ago	✓ SUCCESS Telegram Alert 1 min ago	✓ SUCCESS Notion Save 1 min ago	⏸ PENDING Weekly Report 3 days ago

06 FastAPI AI Agent // python backend

The FastAPI server hosts a LangGraph ReAct agent — an AI agent that reasons and acts in a loop, using tools to fetch, analyze, and store financial intelligence.

API ENDPOINTS

ENDPOINT	PURPOSE
GET /health	Health check — n8n pings this
POST /analyze	Main analysis — called by n8n schedule
POST /api/weekly-report	Generate weekly summary report
GET /docs	Interactive Swagger UI (localhost:8000/docs)

REACT AGENT LOOP

1. **Receive articles from n8n**
   Array of articles from News API, Alpha Vantage, Finnhub
2. **Sentiment analysis via GPT-4**
   Each article scored −1.0 (bearish) to +1.0 (bullish)
3. **Generate embeddings**
   `text-embedding-3-small` converts articles to vectors
4. **Store in Pinecone**
   Vectors saved with metadata for future RAG queries
5. **Return structured response**
   Articles + scores + alerts + recommendations → back to n8n

ENVIRONMENT CONFIGURATION (.ENV)

```
# FastAPI Environment Variables
OPENAI_API_KEY=sk-your-openai-api-key
PINECONE_API_KEY=your-pinecone-api-key
PINECONE_INDEX_NAME=financial-agent
PINECONE_HOST=your-index-host.pinecone.io
```

07 Automated Report Generation // weekly intelligence

Every week, n8n's Schedule Trigger fires the report workflow which queries Pinecone for all articles stored that week, feeds them to GPT-4, and distributes the generated report via Telegram and Notion simultaneously.

WEEKLY REPORT WORKFLOW

⏰ **Weekly Trigger**
Monday schedule

→

POST /api/weekly-report
FastAPI endpoint

→

Pinecone RAG
Query week's articles

→

GPT-4 Report
Generate summary

↓ Return { report, period, totalAlerts, opportunities, risks }

📱 **Telegram**
Send to your chat

and

📓 **Notion Reports DB**
Save with date + period

📋 REPORT CONTAINS

- ✓ GPT-4 written weekly market summary
- ✓ Total alerts triggered that week
- ✓ Opportunities identified (bullish signals)
- ✓ Risks flagged (bearish signals)
- ✓ Time period covered
- ✓ Saved to Notion with searchable date

⚙ REQUIREMENTS TO ENABLE

- → FastAPI running with OpenAI key
- → Pinecone index populated with articles
- → Telegram Bot token in n8n credentials
- → `NOTION_REPORTS_DB_ID` env variable set
- → Weekly Report Trigger set to Active in n8n

08 Dashboard Components // what the UI shows

📈 STATSBAR

Six live metrics across the top: Articles Processed, Alerts Sent, Opportunities, Risk Alerts, Avg Sentiment score, and System Uptime.

🎯 SENTIMENTGAUGE

Visual gauge showing the overall market sentiment score from −1.0 (extremely bearish) to +1.0 (extremely bullish) with a Bullish/Bearish label.

⚡ WORKFLOWSTATUS

Grid of all 8 n8n workflow nodes with their current status (running/success/pending/error) and last execution time.

📰 NEWSFEED

Scrollable list of analyzed financial articles with source badge, sentiment score, ticker symbols, and external link. Sorted by recency.

🔔 ALERTSPANEL

Triggered investment alerts with action type (BUY/SELL/WATCH/HOLD), sentiment score, source, and URL. Also saved to Notion.

💾 DOWNLOADSECTION

Download buttons for the n8n workflow JSON, FastAPI main.py, requirements.txt, and README setup guide directly from the browser.

ROUTE STRUCTURE & ACCESS CONTROL

ROUTE	PAGE	ACCESS	PURPOSE
/login	Login.tsx	Public	Sign in with username + password
/register	Register.tsx	Public	Create account + set investor profile
/	Index.tsx	Protected	Main dashboard — all components
/profile	Profile.tsx	Protected	Edit risk tolerance, interests, Telegram ID

09 Data Storage Map // what lives where

DATA	STORAGE LOCATION	PERSISTENCE	NOTES
User accounts & passwords	n8n SQLite (workflow static data)	Permanent	Survives restarts. SHA-256 hashed passwords only.
Browser session	localStorage	Until logout	{ username, userId, profile } — no password stored
Article embeddings	Pinecone Vector DB (Cloud)	Permanent	Used for RAG personalization & weekly reports
Processed articles	Notion — Articles Database	Permanent	Title, Sentiment, Action, Source, URL, Date
Weekly reports	Notion — Reports Database	Permanent	Full GPT-4 report text + period + metrics
FastAPI analysis results	In-memory (temporary)	Per request	Passed to n8n, not persisted independently

10 Project File Structure // key files

NEW FILES CREATED

FILE	PURPOSE
src/lib/auth.ts	SHA-256 hashing, n8n API calls, localStorage helpers
src/lib/user-management-workflow.json	n8n workflow — register/login/profile webhooks
src/contexts/UserContext.tsx	React context — global auth state
src/components/ProtectedRoute.tsx	Route guard — redirects to /login if no session
src/pages/Login.tsx	Sign-in page
src/pages/Register.tsx	Account creation + investor profile setup
src/pages/Profile.tsx	Edit preferences + sign out
public/favicon.svg	Finance-themed trending-up chart icon

MODIFIED FILES

FILE	CHANGE
src/App.tsx	Added UserProvider, Login/Register/Profile routes, ProtectedRoute
src/pages/Index.tsx	Added user widget in header + personalization filter badges
index.html	Removed Lovable tags, added InvestoraAI title + favicon
vite.config.ts	Removed lovable-tagger plugin
package.json	Renamed to investora-ai, removed lovable-tagger dep
main.py	Changed FastAPI title to "Investora AI"
requirements.txt	Changed == to >= for Python 3.14 compatibility

11 Startup Guide // every time you restart

STEP 1 — DOCKER / N8N

- → Start Docker Desktop (n8n container starts automatically)
- → Open `http://localhost:5678`
- → Find "InvestoraAI User Management" workflow
- → Toggle **Active** if not already on
- → Confirm "Webhook is listening" message

STEP 2 — FASTAPI SERVER

```
# In VS Code terminal
cd public/downloads/fastapi-langchain-agent
venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

You should see "Application startup complete."

STEP 3 — REACT FRONTEND

```
# In a NEW terminal (project root)
npm run dev
```

Open `http://localhost:8080` in your browser

ONE-TIME SETUP ONLY

- ✓ Once `python -m venv venv`
- ✓ Once `pip install -r requirements.txt`
- ✓ Once `npm install`
- ✓ Once Import n8n workflows via Docker CLI
- ✓ Once Fill in all API keys in .env

COMPLETE API KEYS CHECKLIST

KEY / TOKEN	WHERE TO GET	WHERE TO SET	REQUIRED FOR
OPENAI_API_KEY	platform.openai.com	FastAPI .env	Sentiment analysis + report generation
PINECONE_API_KEY	pinecone.io console	FastAPI .env	Storing + querying article embeddings
PINECONE_HOST	Pinecone index page	FastAPI .env	Connecting to your specific index
NEWS_API_KEY	newsapi.org	n8n environment vars	Fetching financial news headlines
ALPHA_VANTAGE_API_KEY	alphavantage.co	n8n environment vars	Market data + pre-built sentiment scores

KEY / TOKEN	WHERE TO GET	WHERE TO SET	REQUIRED FOR
FINNHUB_API_KEY	finnhub.io	n8n environment vars	Real-time financial news
TELEGRAM_BOT_TOKEN	@BotFather on Telegram	n8n Telegram node credentials	Sending alerts + weekly reports
NOTION_API_KEY	notion.so/my-integrations	n8n Notion node credentials	Saving articles + reports to Notion
NOTION_DATABASE_ID	Notion Articles DB URL	n8n Notion node	Target database for articles
NOTION_REPORTS_DB_ID	Notion Reports DB URL	n8n environment vars	Target database for weekly reports

12 Where to View Logs // debugging & monitoring

📋 N8N EXECUTION LOGS (BEST)

Open `http://localhost:5678` → Click your workflow → Click **Executions** tab

Shows every webhook call: input data, node-by-node output, errors in red. Best place to debug register/login issues.

🐳 DOCKER CONTAINER LOGS

```
docker logs n8n --tail 50
# or live:
docker logs n8n -f
```

FASTAPI LOGS

Visible directly in the VS Code terminal where uvicorn is running:

```
INFO: POST /analyze → 200 OK
INFO: Application startup complete.
```

🌐 BROWSER CONSOLE

Press **F12** in your browser → Console tab. Shows fetch errors, auth flow steps, and React runtime errors.

InvestoraAI · v1.0 · Full Stack Financial AI Agent

React + Vite + TypeScript · FastAPI + LangChain + LangGraph · n8n + Docker · OpenAI + Pinecone